Course: BSCpE	**Section: 4-4**
Name/s:	**Date Performed: March 28, 2026**
Anog, Nathaniel D.	**Date Submitted: March 30, 2026**
Catalan, Timothy John O.	**Instructor: Engr. Mon Arjay Malbog**
Dela Cruz, Elleazar F.	
Estandante, Arjhann R.	
Quicay, Mark Florence D.V.	
Reginio, Nataniel P.	
Seraspi, Justin Carl	

Laboratory Activity No. 3

Machine Learning–Based Environmental Monitoring using ESP32 and MicroPython

1. Objective(s):

This laboratory activity introduces students to the integration of machine learning concepts with embedded IoT systems. Students will explore how environmental sensor data can be used to train a machine learning model and how the trained model can be applied to make predictions within a microcontroller-based system.

The activity demonstrates how sensor data collected from an IoT device can be used to develop predictive models that classify environmental conditions. Students will also understand how trained machine learning models can be simplified and deployed in embedded systems for real-time decision making.

Through this activity, students will gain experience in:

- Environmental data acquisition using sensors
- Dataset generation and preparation
- Training a machine learning model using PythonExtracting model parameters for embedded systems
- Applying machine learning predictions in IoT devices
- Monitoring system data through a web interface

2. Intended Learning Outcomes (ILOs):

At the end of the activity student shall able to:

- Explain how machine learning models can be integrated into IoT systems.
- Describe the process of dataset generation and labeling.
- Understand how machine learning models are trained and evaluated.
- Explain how trained models can be deployed to embedded systems.

- Analyze environmental sensor data and interpret predictions generated by the system.
- Understand the role of web interfaces in IoT monitoring systems.

3. Materials

- ESP32 Development Board
- Breadboard
- Jumper wires
- LEDs
- Push buttons
- DHT11 Temperature and Humidity Sensor
- I2C LCD Display (16x2)
- USB cable
- Computer with Python installed
- Thonny IDE

4. Discussion

4.1 Environmental Monitoring Systems

Environmental monitoring systems are widely used to observe and analyze physical conditions such as temperature, humidity, air quality, and atmospheric pressure. These systems are essential in many fields including agriculture, smart buildings, industrial monitoring, and climate research.

In traditional monitoring systems, sensors collect environmental data and transmit it to a central computer or server where it is analyzed. However, modern IoT systems increasingly integrate **intelligent decision-making capabilities directly within embedded devices**.

This allows the system to interpret sensor readings locally and respond immediately to environmental changes.

In this laboratory activity, the ESP32 microcontroller will collect environmental data using the **DHT11 temperature and humidity sensor**. The collected data will then be analyzed using a machine learning model to determine whether the environmental conditions are comfortable or potentially problematic.

4.2 Machine Learning for Environmental Prediction

Machine learning is a field of artificial intelligence that enables computers to learn patterns from data and use those patterns to make predictions.

Unlike traditional programming, where rules are explicitly written by the programmer, machine learning models learn relationships between input variables and outcomes based on training data.

For example, a traditional rule-based system might use the following logic:

If temperature > 30°C then environment is hot

However, this approach is limited because environmental comfort depends on multiple factors. Machine learning models can analyze multiple variables simultaneously and learn complex relationships between them.

In this activity, the machine learning model will use two input variables:

- Temperature
- Humidity

Based on these inputs, the model will classify the environmental condition.

4.3 Dataset Preparation

Machine learning models require datasets for training. A dataset is a collection of data samples that include input values and corresponding labels.

Each data sample in this activity will include:

Temperature	Humidity	Label
23°C	55%	Comfortable
31°C	75%	Hot
28°C	85%	Humid

The label represents the environmental condition associated with the recorded temperature and humidity.

In real-world machine learning systems, datasets are collected from sensors over long periods of time and manually labeled by domain experts. However, collecting thousands of real sensor readings would require significant time.

Therefore, for laboratory purposes, students will generate a synthetic dataset that simulates realistic environmental conditions.

4.4 Machine Learning Model Training

Once the dataset has been prepared, the next step is to train a machine learning model.

During training, the algorithm analyzes the dataset and identifies relationships between input variables and output labels. In this activity, a **logistic regression classifier** will be used.

Logistic regression is commonly used for classification problems because it predicts the probability that an input belongs to a particular class.

The model learns numerical parameters known as **weights** and **bias**.

The prediction equation used by logistic regression is:

$$z = w_1 \times temperature + w_2 \times humidity + bias$$

The result is then converted to a probability using the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

If the resulting probability exceeds a certain threshold, the model predicts that the environment is uncomfortable.

4.5 Machine Learning Deployment in Embedded Systems

Although machine learning models are trained on powerful computers, they can also be deployed in embedded devices by embedding the trained parameters directly into the program.

Microcontrollers such as the ESP32 cannot run large machine learning frameworks, but they can easily compute mathematical equations derived from trained models.

This means the trained weights and bias values obtained during training can be inserted into the ESP32 program. The microcontroller then performs the same prediction calculations using real sensor data.

This process is known as **model inference**.

The advantage of performing inference locally is that the system can make predictions without requiring an internet connection.

5. Procedures

Before starting the main activity, students must prepare the ESP32 to run MicroPython programs and verify that the board is working correctly. In this step, students will install the required software, upload the MicroPython firmware to the ESP32, connect the board using the Thonny IDE, and run a simple LED blinking program.

MicroPython is a lightweight version of the Python programming language designed to run on microcontrollers. Unlike the Arduino environment where programs are compiled before uploading, MicroPython allows programs to be written and executed directly on the device. This makes development easier and faster, especially for beginners.

Students will use Thonny IDE, which provides a simple interface for writing Python code and transferring it to the ESP32.

Part A — Preparing the ESP32 and Running Basic MicroPython Programs

This part ensures that the ESP32 is correctly configured to run MicroPython and verifies that the board is functioning properly.

Step 1 — Install Thonny IDE

Download and install the **Thonny IDE**, which will be used to write and upload MicroPython programs.

Download link:

https://thonny.org

After installation, open the Thonny IDE.

Step 2 — Download MicroPython Firmware for ESP32

Download the MicroPython firmware from the official MicroPython website.

https://micropython.org/download/esp32/

Download the latest firmware file. The filename will look similar to:

```
esp32-20240105-v1.22.bin
```

Save this file on your computer.

Step 3 — Install the Flashing Tool

Open a terminal or command prompt and install the flashing tool using the following command:

```
pip install esptool
```

This tool will allow you to upload the MicroPython firmware to the ESP32.

Step 4 — Connect the ESP32 to the Computer

Connect the ESP32 board to your computer using a USB cable.

Your computer will assign a communication port.

Examples:

Windows

```
COM3
COM4
```

Linux / Mac

```
/dev/ttyUSB0
/dev/ttyACM0
```

Step 5 — Erase the ESP32 Flash Memory

Run the following command in the terminal to erase the flash memory of the ESP32.

Example:

```
esptool.py --port COM3 erase_flash
```

Replace **COM3** with the port assigned to your ESP32.

Step 6 — Upload the MicroPython Firmware

Run the following command to upload the firmware.

```
esptool.py --chip esp32 --port COM3 --baud 460800
write_flash -z 0x1000 esp32-xxxx.bin
```

Replace **esp32-xxxx.bin** with the firmware filename you downloaded earlier.

After the upload process finishes, the ESP32 is ready to run MicroPython.

Step 7 — Configure Thonny to Use ESP32

Open **Thonny IDE**.

Navigate to:

```
Tools → Options → Interpreter
```

Select:

```
MicroPython (ESP32)
```

Then select the COM port corresponding to your ESP32.

Click **OK**.

At the bottom of Thonny you should see the MicroPython terminal:

```
MicroPython v1.22 on ESP32
>>>
```

This indicates that the ESP32 is successfully connected.

Step 8 — Create the LED Test Program

Students will now create a simple LED blinking program to verify that the ESP32 is functioning correctly.

Create a new file in Thonny and enter the following code.

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

while True:

  led.value(1)
  time.sleep(1)

  led.value(0)
  time.sleep(1)
```

This program will turn the LED connected to **GPIO 2** on and off every second.

Step 9 — Upload the Program to ESP32

Upload the program using the following steps:

1. Click

   File → Save As

2. Select

   MicroPython Device

3. Save the file as

   main.py

Saving the file as **main.py** ensures that it runs automatically when the ESP32 starts.

Step 10 — Verify the LED Output

Observe the ESP32 board.

The LED should:

- Turn ON for 1 second
- Turn OFF for 1 second
- Continue blinking repeatedly

If the LED blinks successfully, the ESP32 has been correctly configured.

Part B — Interfacing Buttons and Sensors using MicroPython
In this section, students will learn how to read input devices and sensors using MicroPython.

Step 11 — Button Input

Connect a push button using the following configuration.

Button Pin	ESP32
One side	GPIO15
Other side	GND

Create a new file in Thonny and enter the following code.

```
from machine import Pin

button = Pin(15, Pin.IN, Pin.PULL_UP)
led = Pin(2, Pin.OUT)

while True:

    if button.value() == 0:
        led.value(1)

    else:
        led.value(0)
```

Save the program to the ESP32.

Press the button and observe the LED behavior.

Step 12 — Reading the DHT11 Temperature and Humidity Sensor

Connect the DHT11 sensor using the following configuration.

Sensor Pin	ESP32
VCC	3.3V
GND	GND
DATA	GPIO4

Create a new file and enter the following program.

```
import dht
from machine import Pin
import time

sensor = dht.DHT11(Pin(4))

while True:

    sensor.measure()

    temp = sensor.temperature()
    hum = sensor.humidity()

    print("Temperature:", temp)
    print("Humidity:", hum)

    time.sleep(2)
```

Save and run the program.

Observe the temperature and humidity readings in the Thonny console.

Ensure that the ESP32 prints values similar to the following:

```
Temperature: 28
Humidity: 72
```

If readings appear correctly, the sensor interface is working.

Part C — Machine Learning Model Development for Environmental Prediction

In this part of the activity, students will develop a simple machine learning model that predicts environmental conditions using temperature and humidity values. Unlike the previous section where

programs were uploaded to the ESP32, the machine learning model will first be developed and trained on a **computer using Python.**

Training a machine learning model requires a dataset containing input values and corresponding labels. In real-world applications, such datasets are collected from sensors over long periods of time and manually labeled by experts. However, collecting thousands of real sensor readings would require significant time.

For laboratory purposes, students will generate a **synthetic dataset** that simulates environmental conditions. This dataset will then be used to train a machine learning model. The trained model will produce numerical parameters that will later be embedded in the ESP32 program.

Step 14 — Install Required Python Libraries

Before generating the dataset and training the model, students must install the necessary Python libraries.

Open a terminal or command prompt and run the following commands.

```
pip install pandas
pip install scikit-learn
```

These libraries will be used for dataset processing and machine learning model training.

Step 15 — Generate the Environmental Dataset

Students will now create a Python program that generates a dataset containing temperature and humidity values along with their corresponding environmental labels.

Create a new Python file on your computer named:

generate_dataset.py

Insert the following code.

```
import csv
import random

rows = 50000

with open("environment_dataset_50k.csv","w",newline="") as file:

    writer = csv.writer(file)

```

```
    writer.writerow(["Temperature","Humidity","Label"])

    for i in range(rows):

        temp = random.randint(18,40)
        hum = random.randint(30,95)

        if temp > 30:
            label = "Hot"

        elif hum > 75:
            label = "Humid"

        else:
            label = "Comfortable"

        writer.writerow([temp,hum,label])

print("Dataset generated successfully.")
```

This program generates **50,000 synthetic environmental samples**.

Each row contains:

Temperature	Humidity	Label
24	55	Comfortable
33	70	Hot
27	85	Humid

Step 16 — Run the Dataset Generator

Run the program using the command:

```
python generate_dataset.py
```

After running the script, a file named:

```
environment_dataset_50k.csv
```

will be created.

Open the file using Excel or any spreadsheet software to verify the generated data.

Step 17 — Train the Machine Learning Model

Create a new Python file named:

```
train_model.py
```

Insert the following code.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

data = pd.read_csv("environment_dataset_50k.csv")

X = data[["Temperature","Humidity"]]
y = data["Label"]

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

model = LogisticRegression()

model.fit(X_train,y_train)

accuracy = model.score(X_test,y_test)

print("Model Accuracy:",accuracy)

print("Model Weights:",model.coef_)

print("Model Bias:",model.intercept_)
```

Run the training program using the command:

```
python train_model.py
```

The output will display the model accuracy and learned parameters.

Example output:

```
Model Accuracy: 0.94
Model Weights: [[0.32 0.18]]
Model Bias: [-12.7]
```

Record the weights and bias values, because these will be used in the ESP32 program.

The trained machine learning model predicts environmental conditions using the following equation.

$$z = (w_1 \times Temperature) + (w_2 \times Humidity) + Bias$$

Where:

- w_1 is the temperature weight
- w_2 is the humidity weight
- Bias is a constant offset

The result is converted to probability using the sigmoid function.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

If the probability exceeds 0.5, the system predicts that the environment is uncomfortable.

Step 20 — Preparing the Model for ESP32 Deployment

The ESP32 cannot run the full machine learning training algorithm. Instead, the trained parameters are copied directly into the MicroPython program.

Students should record the values printed from the training program.

Example:

w_temp = 0.32 *w_hum = 0.18* *bias = -12.7*

These parameters will be embedded into the ESP32 code, allowing the microcontroller to perform real-time environmental predictions using sensor data

Step 21 — Create the Machine Learning Prediction Program for ESP32

At this point, we will now create a MicroPython program that performs machine learning predictions using the parameters obtained from the training stage.

Open **Thonny IDE** and create a new file.

Insert the following code.

```
from machine import Pin
import dht
import math
import time

sensor = dht.DHT11(Pin(4))

```

```
# Replace these values with the ones obtained from training
w_temp = 0.32
w_hum = 0.18
bias = -12.7

def sigmoid(x):
    return 1/(1+math.exp(-x))

def predict(temp,hum):

    z = (w_temp * temp) + (w_hum * hum) + bias
    probability = sigmoid(z)

    if probability > 0.5:
        return "Hot/Humid"
    else:
        return "Comfortable"

while True:

    sensor.measure()

    temp = sensor.temperature()
    hum = sensor.humidity()

    prediction = predict(temp,hum)

    print("Temperature:",temp)
    print("Humidity:",hum)
    print("Prediction:",prediction)

    time.sleep(3)
```

This program reads sensor values and applies the machine learning equation derived from the trained model.

Upload the program to the ESP32 using the following steps.

1. Click

   File → Save As

2. Select

   MicroPython Device

3. Save the file as

   ml_prediction.py

Run the program.

Observe the output in the **Thonny console**.

Example output:

```
Temperature: 29
Humidity: 70
Prediction: Comfortable
```

Example Complete Workflow with Server:

```
import network
import socket
import dht
from machine import Pin
import math

ssid = "YOUR_WIFI_NAME"
password = "YOUR_WIFI_PASSWORD"

wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(ssid,password)

while not wifi.isconnected():
    pass

print("Connected:",wifi.ifconfig())

sensor = dht.DHT11(Pin(4))

w_temp = 0.32
w_hum = 0.18
bias = -12.7

def sigmoid(x):
    return 1/(1+math.exp(-x))

def predict(temp,hum):

    z = (w_temp * temp) + (w_hum * hum) + bias
    probability = sigmoid(z)

    if probability > 0.5:
```

```
        return "Hot/Humid"
    else:
        return "Comfortable"

addr = socket.getaddrinfo('0.0.0.0',80)[0][-1]

server = socket.socket()
server.bind(addr)
server.listen(1)

print("Web server running...")

while True:

    client, addr = server.accept()

    request = client.recv(1024)

    sensor.measure()

    temp = sensor.temperature()
    hum = sensor.humidity()

    prediction = predict(temp,hum)

    html = f"""
    <html>
    <head>
    <title>ESP32 Environmental Monitor</title>
    </head>
    <body>
    <h2>Environmental Monitoring System</h2>
    <p>Temperature: {temp} °C</p>
    <p>Humidity: {hum} %</p>
    <p>Prediction: {prediction}</p>
    </body>
    </html>
    """

    client.send("HTTP/1.1 200 OK\n")
    client.send("Content-Type: text/html\n\n")
    client.sendall(html)

    client.close()
```

6. Supplemental Activity

In this supplemental activity, extend the environmental monitoring system developed in the previous parts of the laboratory. The ESP32 will simulate a **smart fan controller** capable of displaying environmental information and fan status through both an LCD display and a web-based monitoring interface.

The system will integrate multiple components including environmental sensors, push buttons, an i2C LCD display, and a local web server. The ESP32 will read temperature and humidity data from the DHT11 sensor and apply the **machine learning model** developed earlier to determine whether the environment is comfortable or uncomfortable.

The system will simulate a fan with four possible operating levels. Two push buttons will be used to control the fan level manually.

One button will increase the fan level, while the second button will decrease it. The fan level should range from **Level 1 to Level 4**.

The LCD display will continuously show the current system information. The information displayed must include the following:

- Temperature reading
- Humidity reading
- Machine learning environmental prediction
- Current fan level

In addition to the LCD display, the ESP32 must host a **local web server**. This web server will allow the system status to be viewed through a web browser connected to the same Wi-Fi network.

The web interface should display the same information shown on the LCD screen.

7. Conclusion:

What conclusion can you make based on the activity?

This laboratory activity demonstrated how machine learning can be integrated into embedded IoT systems using the ESP8266 and MicroPython. It showed how sensor data from the DHT11 can be used to train a model and generate real-time environmental predictions.

A key takeaway is the importance of combining hardware and software, where data collection, model training, and system implementation work together to create intelligent systems. The activity also highlighted how machine learning improves traditional systems by enabling data-driven decisions instead of fixed rules.

Despite the limitations of microcontrollers, deploying trained models directly into the ESP8266 proved that real-time prediction is still possible and efficient. Overall, the activity provided valuable insights into building smart systems and demonstrated the practical applications of IoT and machine learning in real-world scenarios.

GDrive Link for Documentation:

https://drive.google.com/drive/folders/1iv9-bsbNPnuMkyZNvp7U2681rkNBWx2W?usp=drive_link

8. Assessment (Rubric for Laboratory Performance):

Intended Learning Outcomes	Unsatisfactory 1	Satisfactory 2	Exemplary 3	Score
Configure ESP32 with MicroPython environment	Unable to configure the ESP32 or establish communication with the development environment	Able to configure the ESP32 and run MicroPython programs but with guidance or minor errors	Successfully configures ESP32 and runs MicroPython programs independently without errors	
Interface sensors and input devices (LED, button, DHT11)	Unable to correctly connect or operate sensors and input/output devices	Able to interface sensors and devices but with partial functionality or errors	Successfully interfaces sensors and devices with correct operation and outputs	
Generate and prepare dataset for machine learning	Unable to generate the dataset or produces incorrect data	Able to generate dataset but with incomplete structure or errors	Successfully generates a complete dataset suitable for model training	
Train machine learning model using Python	Unable to run or train the machine learning model	Able to train the model but with incorrect interpretation or minor errors	Successfully trains the model and correctly identifies the trained parameters	
Deploy trained model to ESP32 and perform prediction	Unable to implement prediction logic on ESP32	Able to implement prediction but with incorrect output or partial functionality	Successfully implements machine learning prediction using sensor inputs	
Implement ESP32 web server for monitoring system data	Unable to create or access the web server	Able to implement web server but with incomplete display or errors	Successfully implements web server and displays system information correctly	
Other comments/observation:		Total Score		
		RATING = (total score) / 18 x 100%		

Evaluated by:	Date:
_____	_____
Printed Name and Signature of Faculty Member	